COP509

Natural Language Processing

Introduction to Transformers

Prof. Georgina Cosma

Department of Computer Science
Loughborough University

g.cosma@lboro.ac.uk

Learning Outcomes

By the end of this session, you will be able to:

1. Explain **why** Transformers were invented and what problem they solve
2. Understand the key innovation: the **Attention Mechanism**
3. Describe the basic architecture of a Transformer model
4. Explain what **BERT** is and how it differs from earlier models
5. Understand the concept of **pre-training** and **fine-tuning**
6. Know when and why to use Transformer-based models for NLP tasks

Teaching Note

This lecture builds conceptual understanding. The goal is to understand *what* Transformers do and *why* they work, not the mathematical details.

Outline

1. The Journey to Transformers: A Brief History
2. The Problem with Previous Approaches
3. The Key Innovation: Attention
4. Understanding Attention with a Simple Example
5. The Transformer Architecture
6. What is BERT?
7. Pre-training and Fine-tuning
8. Practical Applications and Modern LLMs
9. Sentence Transformers
10. Summary

Part 1

The Journey to Transformers

A Brief History

Where We've Been: The Evolution of NLP

How have we represented text for computers?

What you already know:

- Bag of Words – count words
- TF-IDF – weight by importance
- Word2Vec – words as vectors
- These capture word *meaning*

What was missing:

- Understanding *context*
- Words mean different things in different sentences
- "Bank" = river bank or money bank?
- We need **contextual** understanding

The Limitation of Word Embeddings

Recall: Word2Vec Word2Vec represents each word as a **vector** — a list of numbers that capture the word's meaning.

"king" → [0.8, -0.2, 0.5, ..., 0.3]

"queen" → [0.7, -0.3, 0.6, ..., 0.2]

"car" → [-0.1, 0.8, -0.2, ..., 0.5]

Word2Vec: 300 dimensions

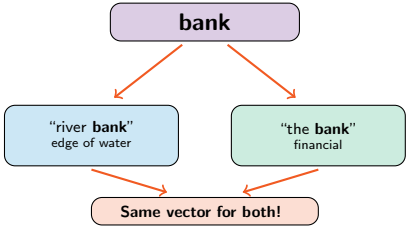
Each number = abstract learned feature, not human-defined

Key terminology:

- **Vocabulary:** The set of all words the model knows (e.g., 3 million words)
- **Dimensionality:** How many numbers per word — fixed for each model (Word2Vec: 300, BERT: 768).
- **Learned features:** The model discovers what each number should capture during training

The Problem: No Context!

Each word has **ONE fixed vector**, regardless of how it's used.

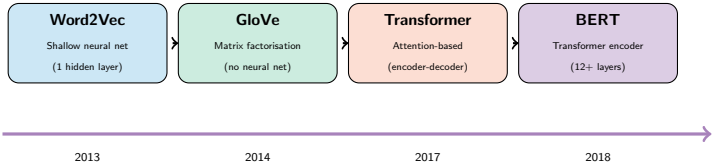


The Solution We Need

A model that looks at the **whole sentence** to understand what each word means *in that context*. The same word should get **different vectors** in different sentences. This is what **BERT** and other **Transformers** provide.

The Evolution of Word Representations

How we got from simple to sophisticated:



All of these are pre-trained models:

Model	Pre-trained by	Trained on	You can...
Word2Vec	Google (2013)	Google News (100B words)	Download and use directly
GloVe	Stanford (2014)	Wikipedia + Common Crawl	Download and use directly
BERT	Google (2018)	Wikipedia + Books	Download, then fine-tune

Word2Vec / GloVe

- **What's pre-trained:** Word embeddings (static vectors)
- **Fine-tuning:** Not typically done
- **Output:** Same vector for "bank" regardless of context

BERT

- **What's pre-trained:** Entire model (contextual vectors)
- **Fine-tuning:** Designed to be fine-tuned for your task
- **Output:** Different vector for "bank" depending on sentence

Key Insight

Pre-training isn't new — Word2Vec and GloVe are also pre-trained. BERT's innovation is the **fine-tuning paradigm**: adapting the entire model to your specific task, not just using frozen word vectors.

Part 2

The Problem with Previous Approaches

The Long-Range Dependency Problem

Real sentences often have words that relate across long distances.

> *"The* **cat***, which had been sleeping on the warm windowsill all afternoon while the rain poured outside, finally* **stretched** *and yawned."*

- The word "**stretched**" relates to "**cat**" – but they are 15 words apart!
- RNNs struggle because they process sequentially
- By the time they reach "stretched", they've "forgotten" about "cat"

The Core Problem

How can a model understand that "stretched" refers to "cat" when they are so far apart in the sentence?

What We Really Need

Imagine you're reading a sentence. You don't read strictly left-to-right!

Our brains:

- Looks at **all** words at once
- Decides which words are **relevant** to each other
- Builds understanding from **relationships** between words

This is exactly what Attention does!

It allows the model to look at **all words simultaneously** and learn which ones are important for understanding each other.

Part 3

The Key Innovation: Attention

In 2017, Google researchers published a groundbreaking paper.

"Attention Is All You Need"

Vaswani et al., 2017

This paper introduced the **Transformer** architecture

The key insight:

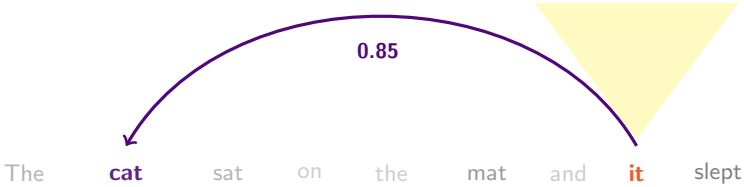
- Throw away RNNs entirely
- Use **only** attention mechanisms
- Process all words **in parallel**
- Let the model **learn** which words to pay attention to

Result: Models that are faster, more accurate, and can handle much longer texts than ever before.

Think of attention like a spotlight.

When understanding a word, the model shines a "spotlight" on other relevant words.

Understanding the word "it":



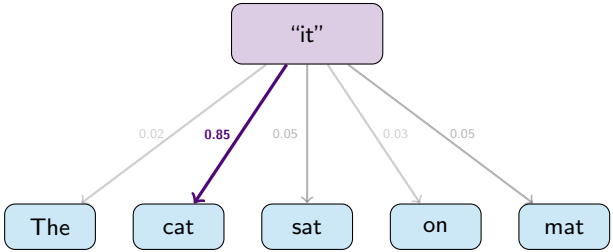
"it" pays strong attention to "cat" – that's what "it" refers to!

Key idea: Each word can "look at" every other word and decide how much attention to pay to each one.

Attention in Simple Terms

For each word, attention answers: "Which other words help me understand this one?"

Query: word we're understanding



Keys: all other words to compare against

Reading the weights:

0.85 → very relevant
0.05 → slightly relevant
0.02 → barely relevant

All weights sum to 1.0

Key Insight

The word "it" gives 85% of its attention to "cat" because that's what "it" refers to! The model **learns** these weights automatically during training.

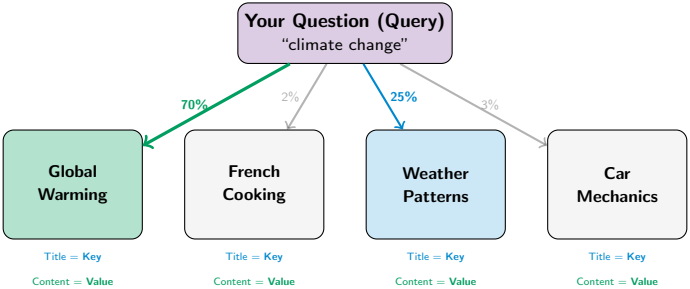
Part 4

Understanding Attention with a Simple Example

Understanding Attention: A Library Analogy

Before diving into the technical details, let's build intuition.

Scenario: You walk into a library looking for information about "climate change".



What happens:

1. You **compare** your question (Query) to each book's title (Key)
2. You decide how **relevant** each book is (attention weights: 70%, 25%, etc.)
3. You **read more** from relevant books (retrieve their Values/contents)

This is exactly how attention works in Transformers!

The Three Components: Query, Key, Value

In Transformers, every word is transformed into three vectors:

Component	Purpose	Library Analogy
Query (Q)	"What information am I looking for?"	Your search question
Key (K)	"What information do I contain?"	Book titles in a library
Value (V)	"Here is my actual content"	The book's contents

How attention uses these components (step by step):

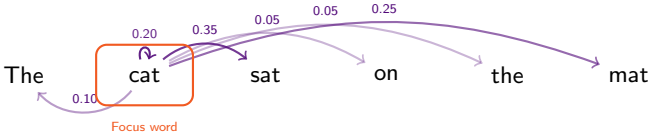
1. **Compare:** Each word's Query is compared against every other word's Key
2. **Score:** This comparison produces similarity scores (how relevant is each word?)
3. **Weight:** Scores are converted to probabilities using **softmax**
   - Softmax ensures all weights are positive and sum to 1.0 (like percentages)
4. **Retrieve:** The output is a weighted combination of all Values

Key Insight

Each word "asks a question" (Query) and "provides an answer" (Value). The attention weights determine whose answers matter most for understanding each word. The model **learns** these Q, K, V transformations during training.

Self-Attention: Words Attending to Each Other

"Self-attention" means every word attends to **every other word** in the same sentence (including itself!).



"cat" attends to all words, with different weights

How to Read the Numbers

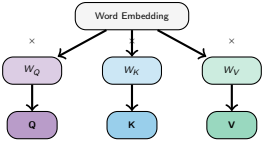
The numbers are **attention weights** (they sum to 1.0). Higher = more relevant.
Here, "cat" pays most attention to "sat" (0.35) and "mat" (0.25) – these words help define what the cat is doing and where!

Every word does this simultaneously – that's why Transformers are **fast**!

The Attention Formula

How are Q, K, V created?

Each word's embedding is transformed using **learned weight matrices**:



The model **learns** W_Q, W_K, W_V during training — discovering what to look for (Q), what to advertise (K), and what to provide (V).

The formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Breaking it down (step by step):

QK^T	**Step 1: Compare.** Multiply Query by transposed Key matrix. Computes similarity scores between every pair of words.
$\div\sqrt{d_k}$	**Step 2: Scale.** d_k = dimensionality of Key vectors (e.g., 64 numbers). Divide by $\sqrt{d_k}$ to prevent large values from saturating softmax.
softmax	**Step 3: Normalise.** Convert scores to probabilities. All weights become positive and sum to 1.0.
$\times V$	**Step 4: Retrieve.** Multiply weights by Values. Output is a weighted average of all words' values.

Example: Why Scaling Matters

If $d_k = 64$ and scores before scaling are [32, 16, 8]:

Without scaling: softmax([32,16,8]) $\approx$ [1.0, 0.0, 0.0]

All attention on one word — gradients vanish!

With scaling: softmax([4, 2, 1]) $\approx$ [0.84, 0.11, 0.05]

Attention spread across words — model can learn

Building Blocks: Vectors in Attention

What is a Word Vector (Embedding)?

A word vector represents a word as a **list of numbers**, where each number captures some aspect of the word's meaning.

"king" → [0.8, -0.2, 0.5, ..., 0.3]

Position 1: royalty score
Position 2: gender score
Position 3: power score
... (learned during training)

What is Dimensionality?

Dimensionality = how many numbers are in the vector. Nothing more!

Word2Vec:	300 numbers → 300-dimensional
BERT:	768 numbers → 768-dimensional

What is d_k? A Concrete Example

d_k = **how many numbers** each Key vector contains.

In BERT, $d_k = 64$. This means every word gets a Key vector with **64 numbers**:

`"cat"` → Key: [0.2, -0.5, 0.8, ..., 0.3] ← **64 numbers**

`"dog"` → Key: [0.3, -0.4, 0.7, ..., 0.1] ← **64 numbers**

`"the"` → Key: [0.1, -0.1, 0.2, ..., 0.2] ← **64 numbers**

Where does 64 come from?

BERT splits its 768-dimensional embedding across 12 attention heads:

$$d_k = 768 \div 12 = 64$$

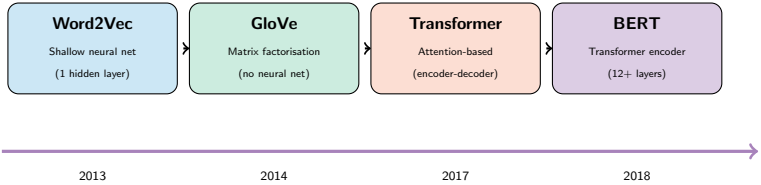
Each head works with smaller 64-number vectors.

Key Takeaway

d_k is just a count of numbers — **not** a count of words! In BERT, every word's Key vector contains exactly 64 numbers. This value appears in the attention formula for scaling.

The Evolution of Word Representations

How we got from simple to sophisticated:



Word2Vec / GloVe (Pre-Transformer)

- Pre-trained word embeddings
- **Static:** same vector for "bank" in every sentence
- Dimensionality: 100–300 (design choice)
- Fast to train and use

BERT (Transformer-based)

- Pre-trained, then fine-tuned
- **Contextual:** different vector for "bank" depending on sentence
- Dimensionality: 768–1024 (architecture design)
- More powerful but computationally heavier

Key Insight

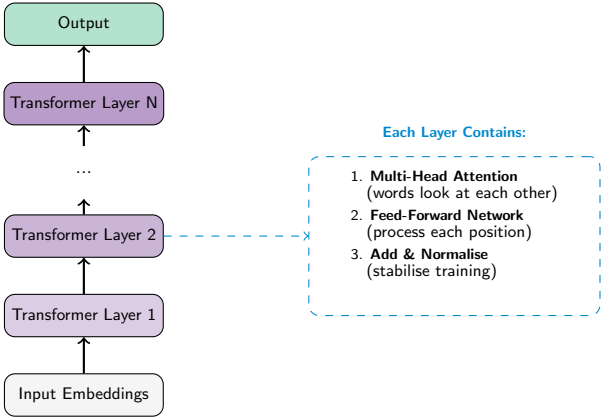
Word2Vec and GloVe are **pre-trained** models too — but BERT introduced **fine-tuning** the entire model for specific tasks, not just using frozen word vectors.

Part 5

The Transformer Architecture

The Big Picture: Transformer Architecture

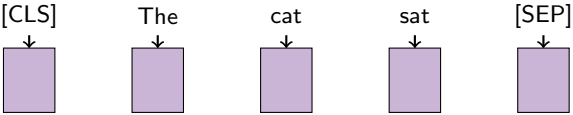
A Transformer is built by stacking layers.



Stacking = Deeper Understanding

Each layer refines the understanding. Layer 1 might capture basic grammar, Layer 6 might capture meaning, Layer 12 might capture complex reasoning. **BERT-base: 12 layers, BERT-large: 24 layers.**

The input to a Transformer is a sequence of embeddings.



[CLS] = Classification token (used for sentence tasks)

[SEP] = Separator token (marks end of sentence)

Each word becomes a vector (e.g., 768 numbers)

Three things are added together for each token:

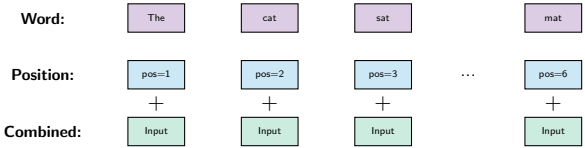
1. **Token Embedding** – the word's meaning (like Word2Vec)
2. **Position Embedding** – where in the sentence (1st, 2nd, 3rd...)
3. **Segment Embedding** – which sentence (for tasks with multiple sentences)

Why Position Matters: Positional Encoding

Problem: Attention treats all words equally – it doesn't know word order!

"The cat sat on the mat" vs "The mat sat on the cat"
Without position information, these look identical to attention!

Solution: Positional Encoding – add position information to each word

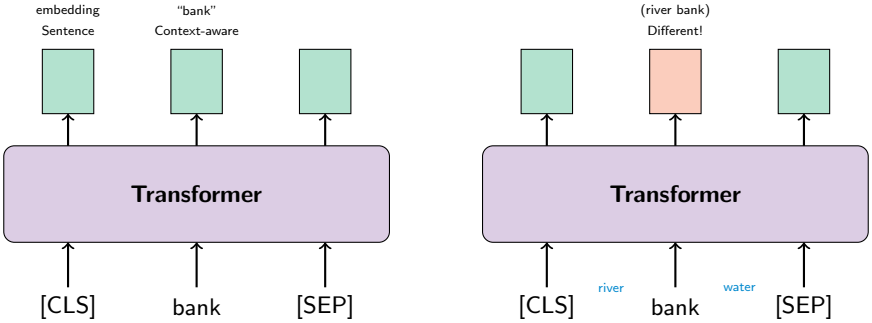


Why This Matters

Now the model knows "cat" is at position 2 and "mat" is at position 6. The sentence "The mat sat on the cat" would have **different** position encodings, so the model can tell them apart!

Key point: This is why Transformers can understand that word order matters!

The output is <u>contextual</u> embeddings – one for each input token.



This Solves the Polysemy Problem!

Remember how Word2Vec gave "bank" the **same** vector regardless of meaning? Transformers give **different** embeddings based on context:

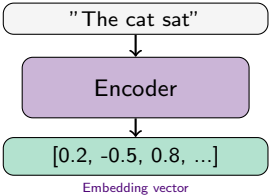
- "money in the bank" → financial institution embedding
- "river bank" → waterside location embedding

What Are Encoders and Decoders?

Before we look at Transformer architectures, let's understand these two core concepts.

Encoder: The "Reader"

An encoder **reads and understands** input text, converting it into a numerical representation (embedding) that captures its meaning.

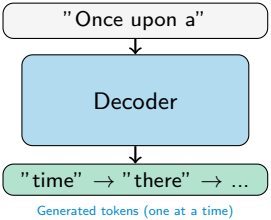


Analogy: Like reading a book and forming a mental summary — you understand it, but you're not writing anything new.

Key property: Sees the **entire input at once** (bidirectional) — can look at words before AND after each position.

Decoder: The "Writer"

A decoder **generates new text** by predicting one word at a time, based on what it has generated so far.



Analogy: Like writing a story word by word — each word depends on what you've already written.

Key property: Only sees **previous tokens** (left-to-right) — cannot look ahead, because future words don't exist yet.

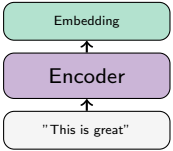
The Key Difference

Encoder = understanding text (input → representation). **Decoder** = generating text (representation → output). Some models use one, some use the other, some use both together.

Three Types of Transformer Architecture

Transformers combine encoders and decoders in different ways for different tasks:

Encoder-Only

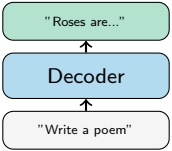


What it does: Reads text and creates a rich numerical representation.
Strengths: Sees entire input at once; excellent for understanding.
Tasks:

- Classification
- Semantic similarity
- Named entity recognition

Examples: BERT, RoBERTa, MPNet

Decoder-Only

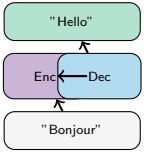


What it does: Generates text one token at a time (autoregressive).
Strengths: Excellent at creative, open-ended generation.
Tasks:

- Text generation
- Chatbots / assistants
- Code completion

Examples: GPT-4, Claude, LLaMA

Encoder-Decoder



What it does: Encoder understands input; decoder generates output.
Strengths: Best when input and output are different forms.
Tasks:

- Translation
- Summarisation
- Question answering

Examples: T5, BART, mBART

Part 6

What is BERT?

BERT: Bidirectional Encoder Representations from Transformers

BERT was released by Google in 2018 and revolutionised NLP.

Bidirectional **E**ncoder **R**epresentations from **T**ransformers

What makes BERT special?

- **Bidirectional:** Looks at words both before AND after each word
- **Pre-trained:** Already learned from massive amounts of text
- **Fine-tunable:** Can be adapted to specific tasks easily

Why "Bidirectional" Matters

Previous models (like GPT-1) only looked left-to-right. BERT looks in **both directions**, giving it better understanding of context.

BERT vs Previous Models

Before BERT: Models read text in one direction only.

Previous models (left-to-right):

The cat **sat** on mat

Only sees context from left

BERT (bidirectional):

The cat **sat** on mat

Sees context from BOTH sides!

This is crucial for understanding!

- "The bank was steep" – need "steep" to know it's a river bank
- "I deposited money in the bank" – need "money" to know it's financial

How BERT Was Pre-trained

BERT learned language from two clever tasks:

Task 1: Masked Language Modelling

Hide some words and predict them:

Original: "The cat sat on the mat"
Masked: "The [MASK] sat on the mat"
BERT predicts: "cat"

This forces BERT to understand context to predict the hidden word.

Task 2: Next Sentence Prediction

Given two sentences, are they consecutive?

Sentence A: "I went to the shop."
Sentence B: "I bought some milk."
BERT predicts: **Yes, consecutive**

This teaches BERT about relationships between sentences.

Training Data

BERT was trained on:

- BookCorpus (800M words from 11,000 books)
- English Wikipedia (2,500M words)

Note: Later research (RoBERTa) showed NSP doesn't help much – MLM is the key task!

BERT Model Sizes

BERT comes in different sizes:

Model	Layers	Hidden Size	Attention Heads	Parameters
BERT-base	12	768	12	110M
BERT-large	24	1024	16	340M

In the tutorial, we'll use DistilBERT:

- A **smaller, faster** version of BERT
- 40% smaller, 60% faster
- Retains 97% of BERT's performance
- Perfect for learning and quick experiments

Why DistilBERT?

For most tasks, DistilBERT gives nearly the same results as BERT but runs much faster and uses less memory – ideal for teaching and prototyping.

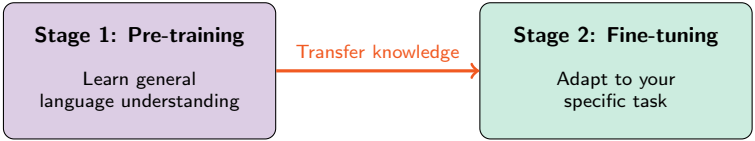
Part 7

Pre-training and Fine-tuning

The Two-Stage Process

The Two-Stage Process

Using BERT involves two stages:



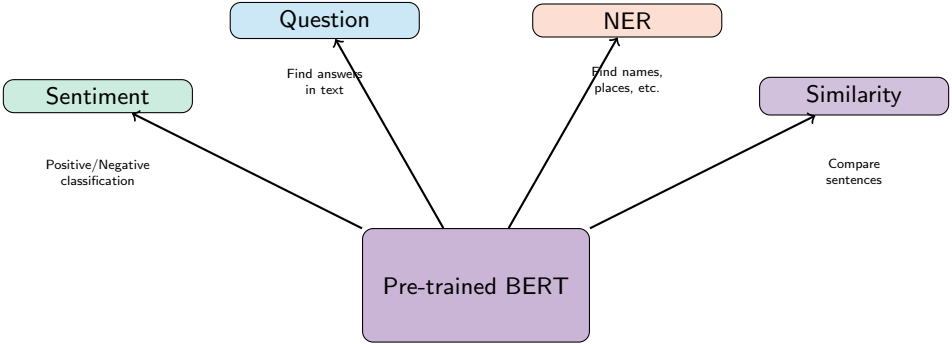
- Done by Google/researchers
- Huge computational cost
- Weeks of training
- You download the result

- Done by you
- Small computational cost
- Minutes to hours
- Small labelled dataset

Good news: You don't need to pre-train! Just download and fine-tune.

Fine-tuning for Different Tasks

The same pre-trained BERT can be fine-tuned for many tasks:



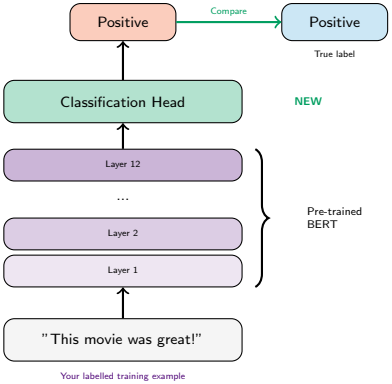
What Does Fine-tuning Require?

Each task only needs a **small labelled dataset** (e.g., 1,000–10,000 examples) because BERT already understands language. Without pre-training, you would need millions of examples to train from scratch.

In today's tutorial: We'll use BERT for sentiment analysis – classifying movie reviews as positive or negative.

What Actually Happens During Fine-tuning?

Fine-tuning adapts a pre-trained model to perform your specific task.



The Fine-tuning Process:

Step 1: Add a classification head
A new layer on top of BERT that outputs your task labels (e.g., Positive/Negative).

Step 2: Feed labelled examples
Pass your training data through the model and compare predictions to true labels.

Step 3: Update weights
Adjust **all** weights (BERT + head) slightly based on errors. This is called backpropagation.

Step 4: Repeat
Process all examples multiple times (epochs) until performance stabilises.

Why "fine"-tuning?

The updates are small — we **nudge** the weights rather than rewrite them. BERT's language knowledge is preserved.

Key Insight

Pre-trained weights give you a head start. Fine-tuning adapts them to your task — that's why it works with only 1,000–10,000 labelled examples instead of millions.

Fine-tuning Step by Step

Here's what happens when you fine-tune for sentiment analysis:

1. **Load pre-trained BERT** – download weights trained by Google/researchers
2. **Add a classification head** – a simple layer that outputs class probabilities
   - Takes the [CLS] token embedding (768 numbers)
   - Outputs 2 numbers: probability of Positive, probability of Negative
3. **Prepare your labelled data** – e.g., movie reviews with Positive/Negative labels
4. **Train for a few epochs** – typically 2–4 epochs is enough!
   - Use a **small learning rate** (e.g., 2e-5) – we're fine-tuning, not training from scratch
   - The model learns to associate language patterns with your labels
5. **Evaluate and use** – your model is now a sentiment classifier!

That's it! The Hugging Face `Trainer` class handles most of this for you.

Why Does Fine-tuning Need So Little Data?

Compare training from scratch vs fine-tuning:

	From Scratch	**Fine-tuning**
What model knows	Nothing	Language structure, grammar, meaning, world knowledge
What it must learn	Everything about language + your task	Only your specific task
Data needed	Millions of examples	Hundreds to thousands
Training time	Days to weeks	Minutes to hours
Compute cost	Very high	Low (laptop/Colab)

The Secret

BERT already understands that "amazing", "wonderful", "loved it" are positive words, and "terrible", "awful", "hated it" are negative. Fine-tuning just teaches it to output a label!

Fine-tuning: Practical Tips

When fine-tuning BERT, keep these in mind:

Hyperparameters:

- **Learning rate:** 1e-5 to 5e-5
  - Too high = destroys pre-trained knowledge
  - Too low = slow/no learning
- **Epochs:** 2–4 usually enough
- **Batch size:** 16 or 32 (limited by GPU memory)

Data tips:

- Balance your classes if possible
- More data = better results
- Quality labels matter!

Common issues:

- **Overfitting:** Model memorises training data
  - Solution: Use validation set, early stopping
- **Catastrophic forgetting:** Model forgets pre-trained knowledge
  - Solution: Lower learning rate

Start simple:

- Use default hyperparameters first
- DistilBERT for quick experiments
- Scale up if needed

Part 8

Practical Applications

What Can BERT Do?

BERT and its variants power many real-world applications:

Text Classification:

- Sentiment analysis
- Spam detection
- Topic classification
- Intent detection

Information Extraction:

- Named Entity Recognition
- Relation extraction
- Key phrase extraction

Question Answering:

- Reading comprehension
- FAQ systems
- Customer support bots

Semantic Understanding:

- Sentence similarity
- Paraphrase detection
- Search and retrieval

Real-World Impact

Google uses BERT for search queries. It helps understand what you're *really* looking for, not just keyword matching.

The BERT Family: Variants and Improvements

Since BERT, many improved models have been developed:

Model	Key Improvement	Best For
DistilBERT	40% smaller, 60% faster	Quick experiments
RoBERTa	Better training procedure	Higher accuracy
ALBERT	Parameter sharing, smaller	Memory-limited cases
ELECTRA	More efficient pre-training	Better with less data
XLNet	Permutation language model	Long documents

And for text generation:

- **GPT** series (GPT-2, GPT-3, GPT-4) – generates text
- **T5** – treats all NLP as text-to-text
- **BART** – combines BERT and GPT ideas

Part 8a

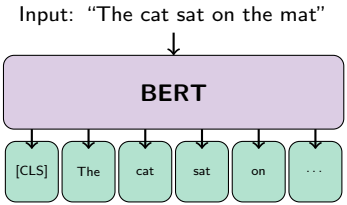
Sentence Transformers

From Word Embeddings to Sentence Embeddings

The Problem: BERT Gives Token Embeddings, Not Sentence Embeddings

Recall: BERT outputs one embedding vector **per token** (word/subword).

But what if you need ONE vector for an entire sentence?



6 vectors! Which one represents the whole sentence?

Common Workarounds:

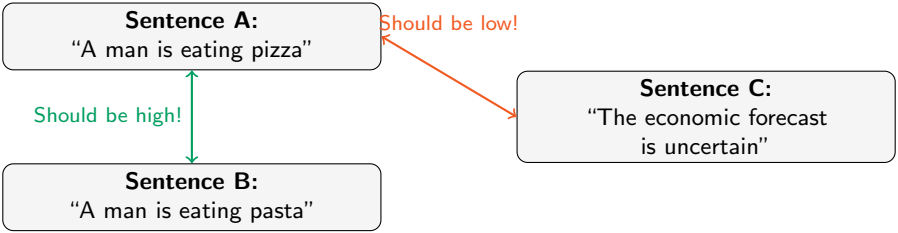
1. **Use the [CLS] token embedding**
   - BERT adds a special [CLS] token at the start of every input
   - This token is used to make predictions in classification tasks
   - Problem: [CLS] was trained for classification, not for measuring similarity
2. **Average all token embeddings**
   - Combine all vectors into one by taking their mean — this is called **pooling**
   - Problem: BERT was not trained to make these averaged vectors meaningful for similarity

The Core Issue

BERT's training objective (predicting masked words) does not encourage similar sentences to produce similar sentence vectors. Two sentences with identical meaning may have very different [CLS] or averaged embeddings — because BERT was never trained to align them.

Why These Approaches Fail: An Example

Consider using BERT embeddings for similarity, search, or retrieval:



What often happens with vanilla BERT embeddings (illustrative):

- Similarity(A, B) $\approx$ 0.75 – seems reasonable
- Similarity(A, C) $\approx$ 0.72 – **Almost the same! This is wrong.**

Note: Exact values vary, but vanilla BERT embeddings often fail to separate unrelated sentences.

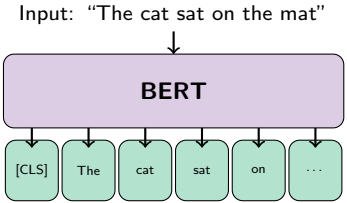
Why This Matters

This affects **semantic search**, **document retrieval**, **clustering**, and **duplicate detection** — all tasks where you need to compare texts. BERT was trained for masked language modelling, not for producing embeddings where similar texts cluster together.

The Problem: BERT Gives Token Embeddings, Not Sentence Embeddings

Recall: BERT outputs one embedding vector **per token** (word/subword).

But what if you need ONE vector for an entire sentence?



6 vectors! Which one represents the whole sentence?

Common Workarounds:

1. **Use the [CLS] token embedding**
   - BERT adds a special [CLS] token at the start of every input
   - This single vector is used as the "sentence representation"
   - Problem: [CLS] was trained for classification, not similarity
2. **Average all token embeddings**
   - **Mean pooling:** add up all vectors and divide by the count
   - E.g., 6 vectors of 768 numbers each $\rightarrow$ 1 vector of 768 numbers
   - Problem: BERT wasn't trained to make these averages meaningful for similarity

The Core Issue

BERT's training objective (predicting masked words) does not encourage similar sentences to produce similar sentence vectors. Two sentences with identical meaning may have very different embeddings — because BERT was never trained to align them.

Popular Sentence Transformer Models

Different models offer trade-offs between speed and quality:

Model Name	Dimensions	Speed	Best For
`all-MiniLM-L6-v2`	384	Very Fast	Quick experiments, prototyping
`all-mpnet-base-v2`	768	Medium	Best quality for general use
`multi-qa-mpnet-base-dot-v1`	768	Medium	Retrieval (uses dot product)
`paraphrase-MiniLM-L6-v2`	384	Very Fast	Paraphrase detection
`all-MiniLM-L12-v2`	384	Fast	Balance of speed and quality

Practical Note

All these models are available on Hugging Face and work with the `sentence-transformers` library. You can switch between them by simply changing the model name!

Using Sentence Transformers: Python Example

The `sentence-transformers` library makes this easy:

```python
# Install: pip install sentence-transformers

from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

# Load a pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Encode sentences to get embeddings
sentences = [
    "The cat sat on the mat.",
    "A cat is sitting on a rug.",
    "The weather is sunny today."
]
embeddings = model.encode(sentences)

# Compute similarity between all pairs
similarities = cosine_similarity(embeddings)
print(similarities)
```

Output:

```
       Sent1  Sent2  Sent3
Sent1 [[ 1.00   0.83   0.21 ]
Sent2  [ 0.83   1.00   0.18 ]
Sent3  [ 0.21   0.18   1.00 ]]
```

High similarity:
Sentences 1 & 2 (both about cats) $\approx$ 0.83

Low similarity:
Sentence 3 (weather) vs others $\approx$ 0.2

Key Insight

Similar meanings $\rightarrow$ similar vectors!

Sentence Transformers vs BERT: When to Use Which

Choose the right tool for your task:

Task	**BERT**	**Sentence Transformers**
Text classification (sentiment, topic)	✓ Best	Can work
Named Entity Recognition	✓ Best	Not suitable
Question Answering (span extraction)	✓ Best	Not suitable
Sentence/document similarity	Not designed for this	✓ **Best**
Semantic search / retrieval	Not designed for this	✓ **Best**
Clustering documents	Not designed for this	✓ **Best**
Finding paraphrases	Not designed for this	✓ **Best**

Key Insight for Your Coursework

If your task involves **comparing documents**, **finding similar passages**, or **semantic search**, use **Sentence Transformers**. If your task involves **classifying** or **extracting information from** a single text, use **BERT**.

Applications of Sentence Transformers

Sentence Transformers enable many practical applications:

Semantic Search:

- Encode all documents once
- Encode user query
- Find documents with highest cosine similarity
- Much better than keyword matching!

Document Clustering:

- Encode all documents
- Apply clustering (e.g., K-means)
- Similar documents group together
- Discover themes automatically

Duplicate Detection:

- Find near-duplicate content
- Paraphrase identification
- Plagiarism detection

Retrieval-Augmented Generation (RAG):

- Encode knowledge base
- Retrieve relevant passages for LLM
- Ground responses in facts
- *You may explore this in coursework!*

Relevance to Your Coursework

Your document search and alignment tasks will benefit from Sentence Transformers – they allow you to find semantically similar content even when the exact words differ.

Quick Reference: BERT vs Sentence Transformers

	BERT	**Sentence Transformers**
Output	One vector **per token**	One vector **per sentence**
Training objective	Masked language modelling	Sentence similarity
Use for similarity?	Not designed for this	Excellent (designed for it)
Use for classification?	Excellent (fine-tune)	Possible but not optimal
Library	`transformers`	`sentence-transformers`
Typical workflow	Fine-tune on labelled data	Encode and compare (often no fine-tuning needed)
Coursework use	Classification tasks	Document similarity, search, retrieval

Key Takeaway

BERT = understanding and classifying text (needs fine-tuning)
Sentence Transformers = comparing and searching text (often works out-of-the-box)
Both are built on the Transformer architecture – they just have different training objectives!

Hugging Face: Your Gateway to Transformers

Hugging Face makes using Transformers easy!

What Hugging Face provides:

- `transformers` library – easy Python interface
- Pre-trained models – download and use immediately
- Model Hub – thousands of models for different tasks and languages
- Datasets – ready-to-use training data
- Documentation and tutorials

In the Tutorial

We'll use `transformers` to load DistilBERT and use it for sentiment analysis – just a few lines of code!

The Bigger Picture: From BERT to ChatGPT

Transformers power today's most impressive AI systems:

BERT-style (Encoders):

- Understanding text
- Classification, NER
- Search, similarity
- *What we learn today*

GPT-style (Decoders):

- Generating text
- ChatGPT, Claude
- Writing, coding
- *Same Transformer core!*

Key insight: Understanding BERT helps you understand ALL modern language models!

When Should You Use Transformers?

Transformers are powerful, but not always the best choice:

USE Transformers when:

- Context matters for meaning
- You have enough data (or can fine-tune)
- Accuracy is more important than speed
- Pre-trained models exist for your language/domain
- Working with complex language tasks

Consider simpler methods when:

- Very limited computational resources
- Extremely simple tasks (keyword matching)
- Interpretability is crucial
- Real-time inference on edge devices
- Training data is extremely limited

Practical Advice

Start with pre-trained models (like DistilBERT) – they're fast to try and often "good enough". Only consider training from scratch for very specialised domains.

Computational Considerations

Transformers require significant resources:

Model	Parameters	Memory (inference)	Typical Use
DistilBERT	66M	∼250 MB	Laptop, quick experiments
BERT-base	110M	∼440 MB	Standard GPU
BERT-large	340M	∼1.3 GB	Good GPU
GPT-3	175B	∼350 GB	Cloud only

Good news for this module:

- DistilBERT runs on **any modern laptop**
- Google Colab provides **free GPU access**
- Pre-trained models mean **no expensive training**
- Hugging Face handles the complexity for you

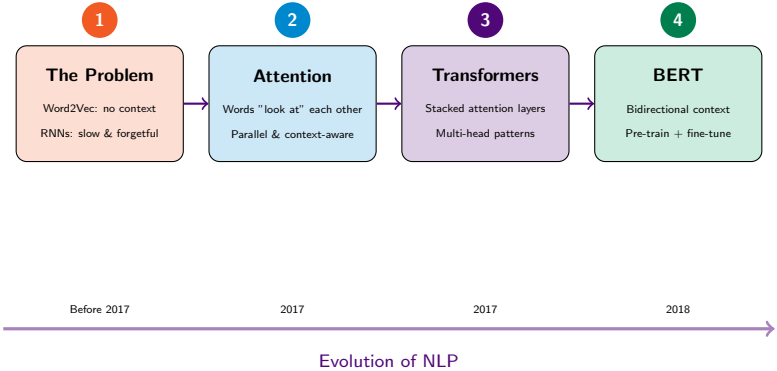
Takeaway: You don't need massive resources to use Transformers effectively!

Part 9

Summary

The Journey from Words to Understanding

Key Takeaways: The Journey to Transformers



The Big Ideas

Attention: Let words decide what's relevant
Transformers: Attention is all you need
BERT: Pre-train once, fine-tune for any task

Why This Matters

Before: Train from scratch for every task
After: Fine-tune in hours with small data
Result: State-of-the-art on almost everything

Practical Tutorials: Applying Sentence Transformers

Both tutorials demonstrate unsupervised applications — no labelled data required!

Tutorial 1: Semantic Search

"Build Semantic Search with LLM Embeddings"
Machine Learning Mastery

- Encode documents with `all-MiniLM-L6-v2`
- Use `NearestNeighbors` for retrieval
- Find relevant documents by meaning, not keywords
- Foundation for RAG systems

Unsupervised: No labels needed — just encode and search!

Tutorial 2: Topic Modelling

"Topic Modeling with BERTopic"
BERTopic Documentation

- Encode documents with Sentence Transformers
- Reduce dimensions with UMAP
- Cluster with HDBSCAN
- Extract topic words with c-TF-IDF

Unsupervised: Topics emerge automatically from clusters!

Key Insight

Pre-trained Sentence Transformers encode semantic meaning, enabling powerful applications **without any task-specific training**. This is a major advantage over traditional approaches like TF-IDF or raw BERT.

Quick Reference: Key Terms

Term	Definition
Transformer	Neural network architecture based entirely on attention
Attention	Mechanism that learns which parts of input are relevant to each other
Self-Attention	Each position attends to all positions in the same sequence
Multi-Head Attention	Multiple attention mechanisms running in parallel
BERT	Bidirectional Encoder Representations from Transformers
Pre-training	Learning general language understanding from large corpora
Fine-tuning	Adapting a pre-trained model to a specific task
[CLS] token	Special token whose output represents the whole sentence
DistilBERT	Smaller, faster version of BERT (97% of performance)
Sentence Transformers	BERT fine-tuned to produce sentence-level embeddings for similarity

References and Further Reading

Original Papers:

- Vaswani et al. (2017), "Attention Is All You Need" – the Transformer paper
- Devlin et al. (2018), "BERT: Pre-training of Deep Bidirectional Transformers"

Accessible Explanations:

- Jay Alammar's "The Illustrated Transformer"
  jalammar.github.io/illustrated-transformer/
- Jay Alammar's "The Illustrated BERT"
  jalammar.github.io/illustrated-bert/

Hugging Face Resources:

- Transformers Documentation: `huggingface.co/docs/transformers`
- Model Hub: `huggingface.co/models`
- Free Course: `huggingface.co/course`

Tutorial Source:

- "A Visual Notebook to Using BERT for the First Time"

Questions?

Prof. Georgina Cosma

g.cosma@lboro.ac.uk